

SCRIPT-CONTROLLED ACL

Restrict Record Access Based on Field Value

ServiceNow • Record-Level Security • Role-Based Access

READ → CREATE → WRITE → DELETE

Platform	ServiceNow
Custom Table	u_institution_details
Security Model	Script-controlled ACL
Project Duration	3 Hours

Prepared as a structured academic project report

1. Executive Summary

This project implements a script-controlled Access Control List (ACL) model in ServiceNow to restrict access to institution/student records according to user roles and the Branch field. The scenario focuses on EEE records while administrators retain full access. The implementation covers READ, CREATE, WRITE and DELETE operations.

2. Project Concept

Core idea: combine role-based authorization with record-level ACL controls so that access is decided before a user can perform a protected operation.

3. Objectives

- Create a controlled ServiceNow environment using a custom institution-details table.
- Create test users and custom roles required by the ACL design.
- Protect record access through READ, CREATE, WRITE and DELETE ACLs.
- Use a script-controlled rule to allow administrator access and apply role-based restrictions.
- Verify the expected access behavior by impersonating users with different role combinations.

4. System Components

ServiceNow Instance: Administrative environment used to configure users, roles, tables and ACLs.

User: Test identity created as EEE User.

Custom Roles: bb1, bb2, bb3 and bb4.

Custom Table: u_institution_details.

Branch: Choice field containing ECE, EEE and CSE.

ACL Engine: Record-level controls for READ, CREATE, WRITE and DELETE.

5. Environment & Configuration

5.1 User and Role Setup

Create the EEE User and custom roles bb1, bb2, bb3 and bb4. Assign the roles for testing the progressively expanded permission model.

5.2 Custom Table

Create Institution Details with the name u_institution_details. Configure Student Roll Number, Student Name, Faculty Name, Branch, Email, Phone Number and Description fields.

6. ACL Architecture

READ → bb1 → Branch = EEE → Script controlled.

CREATE → bb2 → No data condition specified.

WRITE → bb3 → No data condition specified.

DELETE → bb4 → No data condition specified.

7. Script Logic – READ ACL

```
(function () {  
  if (gs.hasRole('admin')) { return true; }  
  if (gs.hasRole('bb1')) { return true; }  
  return false;  
})
```

```
}());
```

The script grants administrators access, then checks for bb1, and finally denies other users.

8. Implementation Workflow

01 Create User → 02 Create Roles → 03 Build Table → 04 Add Fields → 05 Insert ECE/EEE/CSE Records → 06 Configure READ ACL → 07 Add Script → 08 Configure CRUD ACLs → 09 Impersonate → 10 Verify.

9. Security Permission Matrix

bb1: READ.

bb1 + bb2: READ + CREATE.

bb1 + bb2 + bb3: READ + CREATE + WRITE.

bb1 + bb2 + bb3 + bb4: READ + CREATE + WRITE + DELETE.

No required role: Access denied.

Admin: Full access.

10. Testing & Verification

Test 01 – EEE-role user: open protected list and verify intended EEE access.

Test 02 – User without required role: verify access is denied.

Test 03 – Admin: verify all records are accessible.

Test 04 – bb1 + bb2: verify New/Create capability.

Test 05 – bb1 + bb2 + bb3: verify edit capability.

Test 06 – bb1 + bb2 + bb3 + bb4: verify delete capability.

11. Security Design Highlights

Least-Privilege Structure: permissions are separated by operation.

Role + Record Control: READ combines role requirement with Branch-based control.

Administrative Override: administrators receive full access through the READ script.

Operation Isolation: CREATE, WRITE and DELETE use separate ACL records.

Impersonation Testing: behavior is validated using different user contexts.

12. Project Outcome

The completed lab demonstrates how script-controlled READ, WRITE, CREATE and DELETE ACLs enforce record-level security in ServiceNow and protect data from unauthorized viewing, modification, creation and deletion.

13. Conclusion

This project provides a practical demonstration of access governance in ServiceNow. By separating permissions across bb1, bb2, bb3 and bb4 and validating access through ACLs, the implementation creates a structured security model around the Institution Details table.

14. Future Enhancement Ideas

- Extend the branch-security model to additional departments.
- Introduce more granular conditions for user groups and record attributes.
- Add automated positive and negative access tests.

- Create an administrative dashboard for access-control monitoring.
- Document ACL troubleshooting and maintenance procedures.

15. Source-Based Configuration Reference

This documentation is based on the uploaded project/lab source, including its users, roles, table schema, ACL configuration, script logic, verification steps and stated outcome.